1 Brief description

A stochastic cellular automata running a 1k-RAM, 30kHz 6502 CPU pseudo-asynchronously at each cell.

2 Introduction

A cellular automata where each cell runs a virtual 6502, a classic CPU released in 1975. The appeal to retro computing is designed to win a wide audience, to support which we will also develop mobile/serverless app infrastructure to allow mobile code to run on peoples' phones and hop between phones, with mutation and selection. A client viewer layer will allow visual interpretation of virtual memory, as well as local policies e.g. migration to/from server (and/or local neighborhood) or suppression of particular programs. The long-term goal is to obtain a dataset that will allow us to ask whether we can use the tools of population genetics and phylogenetics to study mobile code on a large scale. Subprojects will include the design of various agent-based and cellular automata models from physics, biology, chemistry, and other fields of science and A-Life. To motivate broad adoption we will offer prizes for the most creative and most virally successful programs.

- The paged storage is addressed as a square array of cells with $B = 256$ cells per side and $M = 1024$ bytes per cell, so there are $B^2 = 65,536$ (64k) cells requiring $B^2M = 64$Mb of storage.

- The memory-mapped neighborhood is 7x7 cells around the local origin (0,0). Cell n occupies the space from $1024n$ bytes to $1024n + 1023$. The relationship between cell index, memory location, and offset (x, y) are shown in these tables

  Memory pages.

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	C0..C3	B0..B3	90..93	64..67	74..77	94..97	B4..B7
$y=2$	AC..AF	60..63	50..53	24..27	34..37	54..57	98..9B
$y=1$	8C..8F	4C..4F	20..23	04..07	14..17	38..3B	78..7B
$y=0$	70..73	30..33	10..13	00..03	08..0B	28..2B	68..6B
$y=-1$	88..8B	48..4B	1C..1F	0C..0F	18..1B	3C..3F	7C..7F
$y=-2$	A8..AB	5C..5F	44..47	2C..2F	40..43	58..5B	9C..9F
$y=-3$	BC..BF	A4..A7	84..87	6C..6F	80..83	A0..A3	B8..BB

  Each time execution resumes after an interrupt, the origin is resampled and the neighborhood rotated by a random multiple of 90 degrees. Zero page locations 0xF0-0xF8 are designated as "oriented registers", whose top 6 bits are rotated along with the memory map when a different orientation is sampled. (Note: 0xF9, the storage location for PCHI, is also auto-rotated but is considered controller-reserved.)

  Cell numbers:

	$x=-3$	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$	$x=3$
$y=3$	48	44	36	25	29	37	45
$y=2$	43	24	20	9	13	21	38
$y=1$	35	19	8	1	5	14	30
$y=0$	28	12	4	0	2	10	26
$y=-1$	34	18	7	3	6	15	31
$y=-2$	42	23	17	11	16	22	39
$y=-3$	47	41	33	27	32	40	46

Cell coordinates:

Range	Directions
0	Origin
1..4	N=(0,+1), E=(+1,0), S, W
5..8	NE, SE, SW, NW
9..12	N^2, E^2, S^2, W^2
13..20	N^2E, NE^2, SE^2, S^2E, S^2W, SW^2, NW^2, N^2W
21..24	N^3, E^3, S^3, W^3
25..27	N^2E^2, S^2E^2, S^2W^2, N^2W^2
28..35	N^3E, NE^3, SE^3, S^3E, S^3W, SW^3, NW^3, N^3W
36..43	N^3E^2, N^2E^3, S^2E^3, S^3E^2, S^3W^2, S^2W^3, N^2W^3, N^3W^2
44..48	N^3E^3, S^3E^3, S^3W^3, N^3W^3

- The area from 0xE000-0xEE3F contains some read-only lookup tables for mapping various transformations inside the neighborhood (operations that yield vectors outside the neighborhood return 0xFF)

Table start S	Meaning of $S[i]$
0xE000 $+64j$	Vector addition $v_i + v_j$
0xEC40	Rotation 90° clockwise
0xEC80	Rotation 180°
0xECC0	Rotation 270°
0xED00	Reflection about x-axis
0xED40	Reflection about y-axis
0xED80	X coordinate of v_i plus 3
0xEDC0	Y coordinate of v_i plus 3
0xEE00	See below

  The table from 0xEE00-0xEE3F contains the mapping from (x, y) coordinates to cell indices, with y ascending fastest, starting from cell (-3,-3); so the byte in 0xEE00$+(y+3)+64(x+3)$ is the index of cell with relative offset (x, y) for $-3 \leq x, y \leq 3$.

- We aim to clock the entire board at the rate of a 1981 BBC Micro (2Mhz), so each cell updates at 30.5Hz. (This is a lower bound. There is no reason you can't run the board faster.)

- Interrupts arrive as an (approximately) Poisson process with an average rate of 1 per 4,096 cycles.

Interrupt model. Pre-emptive scheduling is conceptually an IRQ (which the 6502 program can mask using SEI/CLI) followed by an NMI (which it cannot). Memory writeback happens, if the program allows it, between the IRQ and the NMI. Setting the I flag (SEI) thus creates an atomic transaction: writes accumulate until a BRK commits them (software interrupt), and are reverted if a timer interrupt fires instead.

A timer interrupt is handled as follows:

  - If the interrupt disable flag (I) is set, all writes made since the last interrupt are reverted (atomic abort). Otherwise:
  - The B flag (bit 4) is cleared in P.
  - CPU registers are written to the last seven bytes of zero page (0xF9–0xFF).
  - The memory-mapped neighborhood is copied from RAM to storage (un-rotating oriented registers at 0xF0–0xF9).
  - A new origin cell (i, j) and orientation is randomly sampled. The memory-mapped neighborhood of that cell is loaded from storage to RAM.
  - The last four bytes of zero page are overwritten with pseudorandom numbers.
  - Optional (via the `hasCompass` hyperparameter): the scheduler writes the current orientation vector (left-shifted by two bits) to 0xFA, so programs can detect their orientation and adjust their behavior accordingly. If `hasCompass` is disabled, the scheduler writes zero to 0xFA instead.
  - CPU registers are restored from the last seven bytes of zero page (prior to their being overwritten).

- A BRK software interrupt costs 7 CPU cycles, matching the real NMOS 6502 (2 to fetch opcode + operand, 3 to push PC and P to stack, 2 to read the IRQ vector). Undocumented opcodes are treated as BRK 0 at the same 7-cycle cost. Key differences from timer interrupts:
  - The I flag is ignored: writes always commit after BRK (the "NMI" is unconditional).
  - The operand byte b (the byte following BRK) is examined *before* registers are saved. Copy and swap operations (below) use the cell's pre-interrupt state. This means a BRK copy produces a child that inherits the *previous* scheduling's saved registers, not the post-BRK state.
  - The B flag (bit 4) is set in the saved P, following the 6502 convention that BRK and PHP set B while IRQ and NMI clear it. This enables fork detection: after a BRK copy, the child inherits B=0 (pre-BRK state) while the parent gets B=1. Programs can read byte 0xFB and test bit 4.

– BRK with operand 0 resets PC to 0x0000 (preventing zero-filled cells from marching their PC into the RNG region). All other operands advance PC past the BRK+operand as usual.

– The operand byte b selects an operation: The memory-mapped neighborhood has $N^2 = 49$ cells. The 5 inner cells (origin plus 4 cardinal neighbors, indices 0–4) may be used as swap sources.

Operand b	Operation
0	Resets PC to 0x0000. Yields to scheduler.
$1 \leq b < 245$	Swap cells: source = $\lfloor b/49 \rfloor$ (cells 0–4), destination = $b \bmod 49$ (cells 0–48). Self-swaps (src = dest) update move times but do not copy data. Feature-gated by `implementsMove`. Yields to scheduler.
$245 \leq b \leq 252$	Noisy copy: origin cell is copied to cell $b - 244$ (cells 1–8), subject to the noise model (see below). Feature-gated by `implementsCopy`. Yields to scheduler.
253	Sync interrupt request: X and Y registers specify the period in cycles (X = low byte, Y = high byte). The next interrupt for this cell is scheduled at the nearest future absolute multiple of the period, computed from the global board time. Feature-gated by `implementsSync`. Yields to scheduler.
254	Async interrupt request: X and Y registers specify the delay in cycles (X = low byte, Y = high byte). The next interrupt for this cell is scheduled after that many cycles from the current time. Feature-gated by `implementsAsync`. Yields to scheduler.
255	Reserved (no operation). Yields to scheduler.

If a BRK operand's feature flag is not enabled (e.g. `implementsMove` is false), the BRK simply yields to the scheduler with no effect.

Noisy copy model

The noisy copy operation (operands 245–252) copies all 1024 bytes of the origin cell to the destination. Each bit is independently resampled with probability ε (`pBitNoise`), and faithfully copied with probability $1 - \varepsilon$. When a bit is resampled, it becomes 0 with probability q (`pBitNoiseZero`) and 1 with probability $1 - q$. The default is $\varepsilon = 1/2048$ and $q = 0.5$ (fair coin), giving approximately 1 bit error per 256-byte page copied ($\sim$4 errors per cell). Setting $q = 1$ gives pure erasure noise (all resampled bits become zero).

LDA/STA writes (the normal 6502 store instructions) are always exact—the noise applies only to BRK noisy copies.

Board hyperparameters

The controller accepts a `boardParams` dictionary with the following defaults:

Parameter	Default	Description
`pBitNoise`	1/2048	Per-bit resampling probability on BRK noisy copy (ε)
`pBitNoiseZero`	0.5	P(resampled bit = 0); 0.5 = fair coin, 1 = erasure (q)
`nSwapCycles`	0	Cycle budget for BRK copy/swap; fails if fewer cycles remain before next interrupt (0 = no limit)
`hasCompass`	false	Write orientation to $FA (shifted left 2 bits)
`implementsMove`	true	Enable BRK 1–244 swap operations
`implementsCopy`	true	Enable BRK 245–252 noisy copy
`implementsSync`	false	Enable BRK 253 sync interrupt request
`implementsAsync`	false	Enable BRK 254 async interrupt request

The guiding principle when considering adding more software interrupts, or expanding the OS in any way, should be to not add any functionality that can't be justified as "physics".

  - After any copy or swap, control returns to the scheduler.

- A software interrupt due to a bad instruction is handled like BRK 0 (PC resets to 0, control returns to scheduler).

3 Randomness

Several aspects of the VM rely on randomness: the scheduling order and orientation of cells, the inter-interrupt timing, the four pseudorandom bytes written to zero-page addresses 0xFC–0xFF, and the bit-noise in the noisy copy operation.

In the reference implementation, all randomness is generated by a single deterministic pseudorandom number generator (a Mersenne Twister seeded at initialization). This makes simulations fully reproducible given the same seed, which is valuable for debugging, regression testing, and scientific reproducibility.

However, nothing in the VM specification requires deterministic randomness. Hardware or software implementations may substitute true random number generators (e.g. hardware entropy sources) for any or all of these random processes. The system's behavior should be qualitatively the same: the "physics" of the board—scheduling fairness, copy noise, and the statistical properties of the RNG bytes visible to programs—depend only on the *distribution* of the random variables, not on their deterministic reproducibility.

Programs should not rely on correlations between successive RNG values or between the RNG and the scheduling order. A correct program must work under any source of randomness that satisfies the specified distributions.

4 Visualization Conventions

The following conventions describe how a cell may expose information to external viewers (debuggers, phone UIs, web dashboards). **These are conventions, not part of the VM specification.** The VM does not read or interpret these bytes; programs are free to use them for code or data. However, viewers and tooling may render cells based on this layout, so programs that follow the convention get richer visualization for free.

Offset	Size	Convention
0x3C0–0x3DF	32 bytes	16×16 monochrome bitmap (1 bit/pixel, MSB first)
0x3E0–0x3FB	28 bytes	ASCII display name
0x3FC–0x3FE	3 bytes	Reserved
0x3FF	1 byte	Hue (0–255 maps to 0–360° HSV)

Hue byte. The byte at 0x3FF sets the cell's color. The value 0–255 is interpreted as an HSV hue (0–360°). A nonzero hue is rendered at full saturation with brightness scaled by recent activity. Programs that write a hue byte get colored visualization for free; a zero hue falls back to the default activity-based heat coloring.

Monochrome bitmap. 32 bytes at 0x3C0–0x3DF encode a 16×16 pixel bitmap (1 bit per pixel, MSB first, row-major). This is available for detail views or cell inspectors. The hue byte provides the color; the bitmap provides the shape.

Display name. The 28 bytes at 0x3E0–0x3FB are interpreted as an ASCII string. The reference web app parses this as `[cssColor]:[iconifyIconName]` (e.g. `orange:bee`, `red:sword`). If no colon is present, the string is treated as an Iconify icon name from the `game-icons` set.

For SokoScript and other high-level languages: these conventions provide a natural way to expose cell type and state visually. A compiled SokoScript program could write its type name to 0x3E0, set a hue byte, and render its state as a bitmap, giving viewers a rich display without modifying the VM.

For the phone game: the PWA coin miner uses the hue byte for cell color when present, falling back to the overview color (computed from write/move recency via HSV mapping). The monochrome bitmap is available for future detail views.

5 Undocumented NMOS 6502 Opcodes

The NMOS 6502 has 105 opcodes not defined in the official instruction set. In 6502life, all 256 opcodes have defined behavior, divided into three categories:

5.1 Stable opcodes (faithfully emulated)

These combine documented operations in predictable ways and are well-understood from decades of reverse engineering:

- **LAX** (LDA + LDX): loads both A and X from memory.
- **SAX**: stores A & X to memory.
- **DCP** (DEC + CMP): decrements memory, then compares result with A.
- **ISC/ISB** (INC + SBC): increments memory, then subtracts from A.
- **SLO** (ASL + ORA): shifts left, then ORs with A.
- **RLA** (ROL + AND): rotates left, then ANDs with A.
- **SRE** (LSR + EOR): shifts right, then XORs with A.
- **RRA** (ROR + ADC): rotates right, then adds to A.
- **ANC**: AND immediate, copies bit 7 to carry.
- **ALR**: AND immediate, then LSR accumulator.
- **ARR**: AND immediate, then ROR with special flag handling.
- **AXS/SBX**: (A & X) − immediate → X.

These use the same addressing modes and cycle counts as their documented equivalents. Programs that exploit undocumented opcodes (common in C64 demo scenes) will execute correctly.

5.2 JAM/HLT opcodes (halt the cell)

Opcodes $02, $12, $22, $32, $42, $52, $62, $72, $92, $B2, $D2, $F2 freeze the CPU on real hardware. In 6502life, they set a per-cell "halted" flag. A halted cell does not execute until its memory is overwritten by a copy operation. This provides a natural "death" mechanism: random memory that decodes as a JAM instruction permanently stops that cell, creating selective pressure for programs that avoid JAM bytes.

5.3 Unstable opcodes (NOP with correct timing)

These depend on analog bus state and vary between physical chips. In 6502life they are treated as NOPs that consume the correct number of bytes and cycles but have no other effect. This includes XAA/ANE ($8B), AHX/SHA ($93, $9F), TAS/SHS ($9B), SHY ($9C), SHX ($9E), LAS ($BB), and various undocumented NOP variants with 1–3 byte, 2–5 cycle timing.

6 Terminology

The following terms are used consistently throughout the specification, implementation, and IDE documentation. Bold terms are normative.

Term	Definition
Board Storage	The persistent $B \times B \times M$ byte array holding all cell data—analogous to peeking at a hard drive. Running programs cannot see Board Storage directly; they see the Memory Map instead. The IDE Board Pane shows Board Storage in fixed, untranslated/unrotated coordinates.
Stored Cell	A contiguous 1024-byte (M-byte) block within Board Storage, addressed by grid coordinates (i, j).
Scheduler	The (pseudo)code that picks cells for execution, samples orientations and timer durations, and controls the running of 6502 code. Never visible to the programs themselves.
Active Cell	The Stored Cell currently scheduled for execution (coordinates $i_{\text{orig}}, j_{\text{orig}}$).
Active Frame	The Active Cell together with its sampled orientation. The Active Frame determines how the 7×7 neighborhood is mapped into the Memory Map.
Next IRQ Time	The cycle count at which the next timer interrupt will fire. Visible only to the Scheduler, not to programs.
Memory Map	The 49-cell (N^2-cell) RAM address space that the running program actually sees. Board Storage cells are loaded into the Memory Map in spiral order around the Active Cell, rotated by the Active Frame's orientation.
Currently Mapped Cells	The 49 Stored Cells that have been loaded into the Memory Map for the current Active Frame.
Register State	The 6502 CPU registers (A, X, Y, S, P, PC) during execution.
Stored Register State	The zero-page locations ($F9–$FF) in a Stored Cell (or its Memory Map image) that preserve the Register State between scheduling quanta.
Active Context	The upcoming bytes of the Memory Map in the 6502's execution pipeline—the instruction stream starting at the current PC.
Cursor Cell	(IDE) The Stored Cell that owns the memory address corresponding to the current IDE Map Cursor position.

IDE pane mapping:

- The **IDE Map Pane** shows Board Storage restricted to the Currently Mapped Cells. Because the mapping is known, the **IDE Map Cursor** always corresponds to a unique address in the Memory Map, displayed on the pane boundary.

- The **IDE Disassembler Pane** shows the Active Context by default, but can be pointed at any address in the Memory Map.
- The **IDE Board Pane** (minimap) shows a fixed, untranslated/unrotated view of Board Storage. The Cursor Cell and the Active Cell are highlighted.

7 Links

- Avida (Ofria and Wilke, 2004; Ortega et al., 2023) `https://en.wikipedia.org/wiki/Avida_(software)`
- Core War `https://en.wikipedia.org/wiki/Core_War`
- The BBC Micro Bot `https://mastodon.me.uk/@bbcmicrobot`

References

Ofria, C. and Wilke, C. O. (2004). Avida: a software platform for research in computational evolutionary biology. *Artif Life*, 10(2):191–229.

Ortega, R., Wulff, E., and Fortuna, M. A. (2023). Ontology for the Avida digital evolution platform. *Sci Data*, 10(1):608.